

2.5. NVCC: The NVIDIA CUDA Compiler

The NVIDIA CUDA Compiler `nvcc` is a toolchain from NVIDIA for compiling CUDA C/C++ as well as PTX code. The toolchain is part of the [CUDA Toolkit](#) and consists of several tools, including the compiler, linker, and the PTX and Cubin assemblers. The top-level `nvcc` tool coordinates the compilation process, invoking the appropriate tool for each stage of compilation.

`nvcc` drives offline compilation of CUDA code, in contrast to online or Just-in-Time (JIT) compilation driven by the CUDA runtime compiler `nVRTC`.

This chapter covers the most common uses and details of `nvcc` needed for building applications. Full coverage of `nvcc` is found in the [nvcc documentation](#).

2.5.1. CUDA Source Files and Headers

Source files compiled with `nvcc` may contain a combination of host code, which executes on the CPU, and device code that executes on the GPU. `nvcc` accepts the common C/C++ source file extensions `.c`, `.cpp`, `.cc`, `.cxx` for host-only code and `.cu` for files that contain device code or a mix of host and device code. Headers containing device code typically adopt the `.cu.h` extension to distinguish them from host-only code headers `.h`, `.hpp`, `.hh`, `.hxx`, etc.

File Extension	Description	Content
<code>.c</code>	C source file	Host-only code
<code>.cpp</code> , <code>.cc</code> , <code>.cxx</code>	C++ source file	Host-only code
<code>.h</code> , <code>.hpp</code> , <code>.hh</code> , <code>.hxx</code>	C/C++ header file	Device code, host code, mix of host/device code
<code>.cu</code>	CUDA source file	Device code, host code, mix of host/device code
<code>.cu.h</code>	CUDA header file	Device code, host code, mix of host/device code

2.5.2. NVCC Compilation Workflow

In the initial phase, `nvcc` separates the device code from the host code and dispatches their compilation to the GPU and the host compilers, respectively.

To compile the host code, the CUDA compiler `nvcc` requires a compatible host compiler to be available. The CUDA Toolkit defines the host compiler support policy for [Linux](#) and [Windows](#) platforms. Files containing only host code can be built using either `nvcc` or the host compiler directly. The resulting object files can be combined with object files from `nvcc` which contain GPU code at link-time.

The GPU compiler compiles the C/C++ device code to PTX assembly code. The GPU compiler is run for each virtual machine instruction set architecture (e.g. `compute_90`) specified in the compilation command line.

Individual PTX code is then passed to the `ptxas` tool, which generates Cubin for the target hardware ISAs. The hardware ISA is identified by its [SM version](#).

It is possible to embed multiple PTX and Cubin targets into a single binary Fatbin container within an application or library so that a single binary can support multiple virtual and target hardware ISAs.

The invocation and coordination of the tools described above are done automatically by `nvcc`. The `-v` option can be used to display the full compilation workflow and tool invocation. The `-keep` option can be used to save the intermediate files generated during the compilation in the current directory or in the directory specified by `--keep-dir` instead.

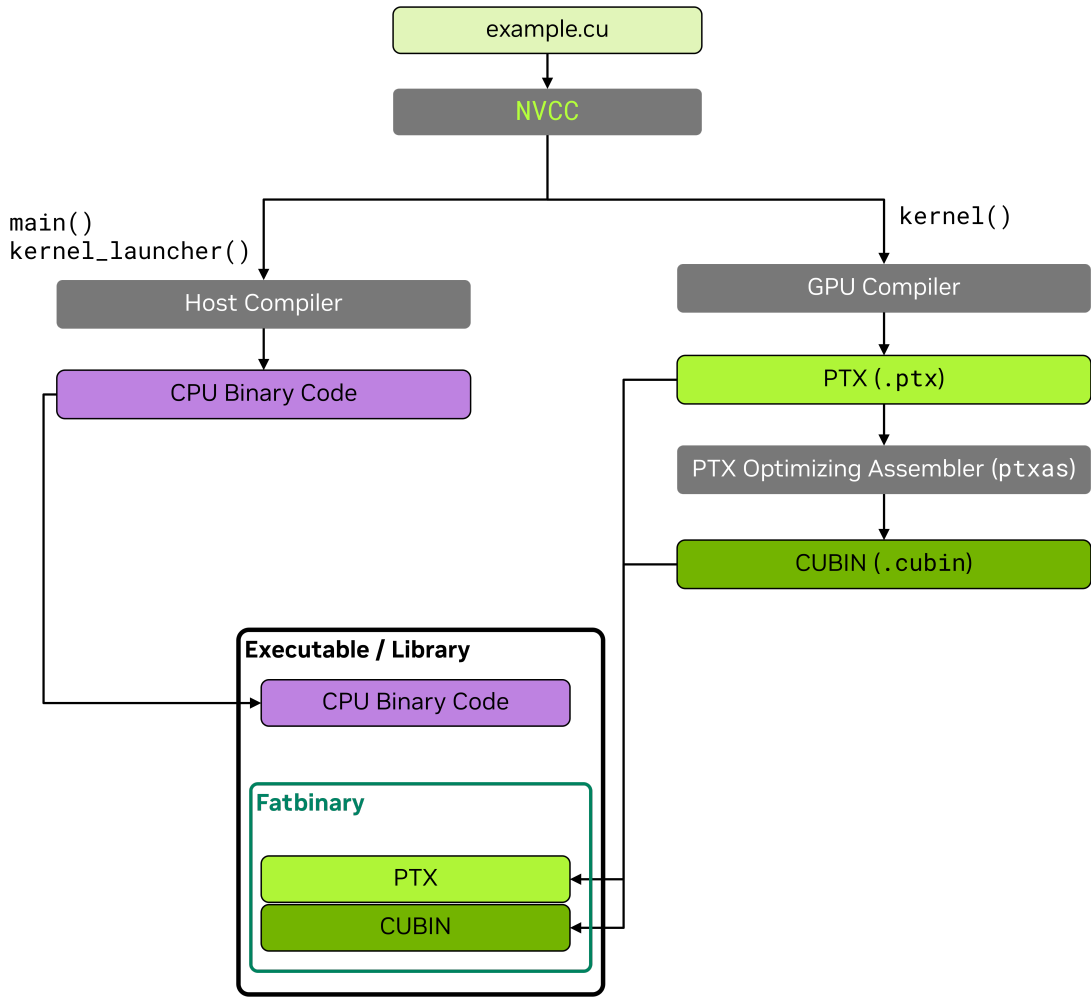
The following example illustrates the compilation workflow for a CUDA source file `example.cu`:

```
// ----- example.cu -----
#include <stdio.h>
__global__ void kernel() {
    printf("Hello from kernel\n");
}

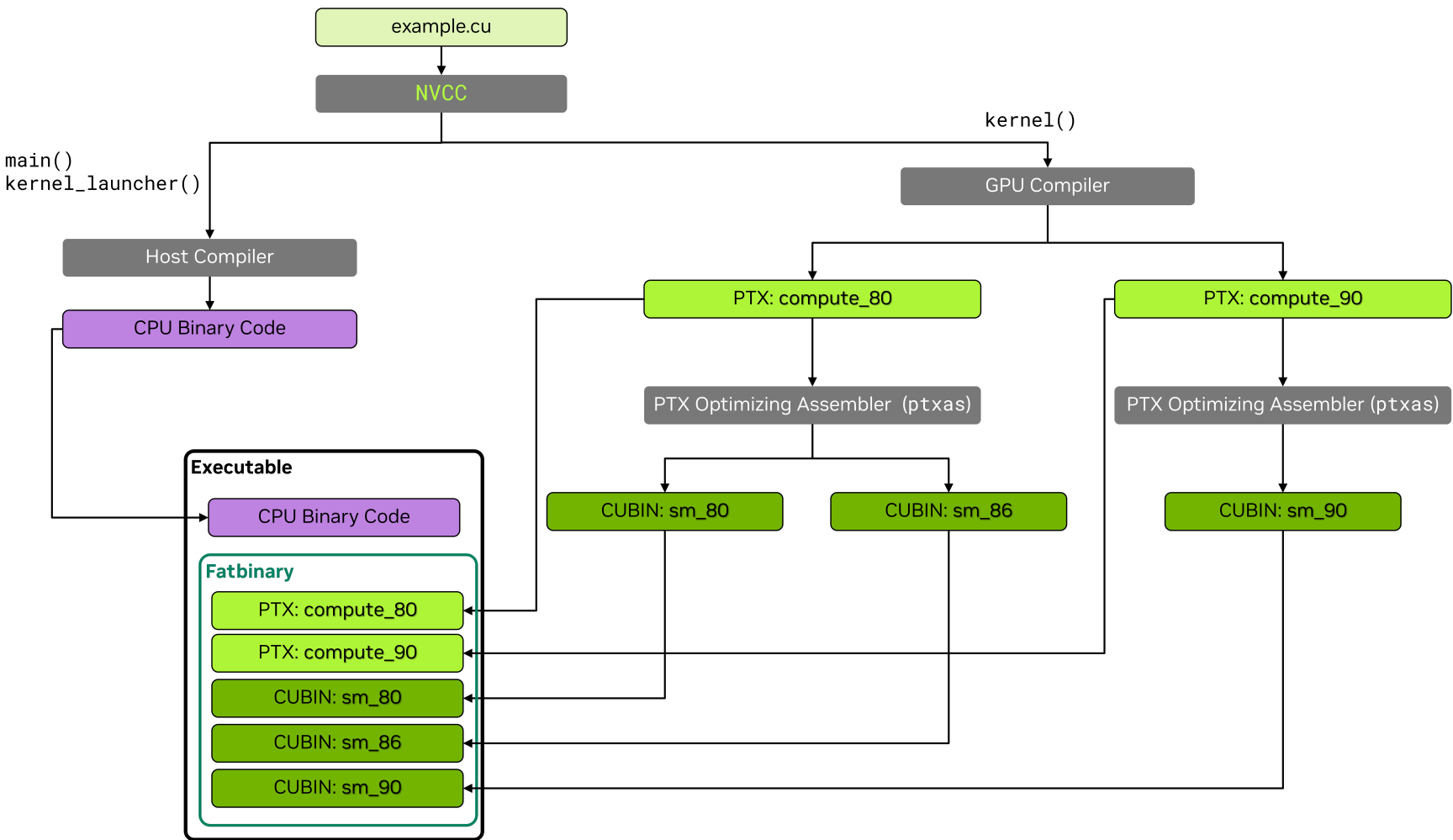
void kernel_launcher() {
    kernel<<<1, 1>>>();
    cudaDeviceSynchronize();
}

int main() {
    kernel_launcher();
    return 0;
}
```

`nvcc` basic compilation workflow:



`nvcc` compilation workflow with multiple PTX and Cubin architectures:



A more detailed description of the `nvcc` compilation workflow can be found in the [compiler documentation](#).

2.5.3. NVCC Basic Usage

The `basic command to compile a CUDA source file` with `nvcc` is:

```
nvcc <source_file>.cu -o <output_file>
```

`nvcc` `accepts common compiler flags` used `for` specifying `include directories` `-I <path>` and `library paths` `-L <path>`, linking against other libraries `-l<library>`, and defining macros `-D<macro>=<value>`.

```
nvcc example.cu -I path_to_include/ -L path_to_library/ -lcublas -o <output_file>
```

2.5.3.1. NVCC PTX and Cubin Generation

By default, `nvcc` `generates PTX and Cubin for the earliest GPU architecture` (lowest `compute_XY` and `sm_XY` version) supported by the CUDA Toolkit to `maximize compatibility`.

- The `-arch` `option` can be used to generate PTX and Cubin for a specific GPU architecture.
- The `-gencode` `option` can be used to generate PTX and Cubin for multiple GPU architectures.

The complete list of supported virtual and real GPU architectures can be obtained by passing the `--list-gpu-code` and `--list-gpu-arch` flags respectively, or by referring to the [Virtual Architecture List](#) and the [GPU Architecture List](#) sections within the `nvcc` documentation.

```
nvcc --list-gpu-code # list all supported real GPU architectures
nvcc --list-gpu-arch # list all supported virtual GPU architectures
```

```
nvcc example.cu -arch=compute_<XY> # e.g. -arch=compute_80 for NVIDIA Ampere GPUs and later
                                     # PTX-only, GPU forward compatible

nvcc example.cu -arch=sm_<XY>       # e.g. -arch=sm_80 for NVIDIA Ampere GPUs and later
                                     # PTX and Cubin, GPU forward compatible

nvcc example.cu -arch=native         # automatically detects and generates Cubin for the current GPU
                                     # no PTX, no GPU forward compatibility

nvcc example.cu -arch=all            # generate Cubin for all supported GPU architectures
                                     # also includes the latest PTX for GPU forward compatibility

nvcc example.cu -arch=all-major      # generate Cubin for all major supported GPU architectures, e.g. sm_80, s
                                     # also includes the latest PTX for GPU forward compatibility
```

More advanced usage allows PTX and Cubin targets to be specified individually:

```
# generate PTX for virtual architecture compute_80 and compile it to Cubin for real architecture sm_86, keep
nvcc example.cu -arch=compute_80 -gpu-code=sm_86,compute_80 # (PTX and Cubin)

# generate PTX for virtual architecture compute_80 and compile it to Cubin for real architecture sm_86, sm_89
nvcc example.cu -arch=compute_80 -gpu-code=sm_86,sm_89      # (no PTX)
nvcc example.cu -gencode=arch=compute_80,code=sm_86,sm_89 # same as above

# (1) generate PTX for virtual architecture compute_80 and compile it to Cubin for real architecture sm_86,
# (2) generate PTX for virtual architecture compute_90 and compile it to Cubin for real architecture sm_90
nvcc example.cu -gencode=arch=compute_80,code=sm_86,sm_89 -gencode=arch=compute_90,code=sm_90
```

The full reference of `nvcc` command-line options for steering GPU code generation can be found in the [nvcc documentation](#).

2.5.3.2. Host Code Compilation Notes

Compilation units, namely a source file and its headers, that **do not contain device code** or symbols can be compiled directly with a host compiler. **If any compilation unit uses CUDA runtime API functions**, the application **must be linked with the CUDA runtime library**. The CUDA runtime is **available as both a static and a shared library**, `libcudart_static` and `libcudart`, respectively. **By default**, `nvcc` **links against the static CUDA runtime library**. To use the shared library version of the CUDA runtime, pass the flag `--cudart=shared` to `nvcc` on the compile or link command.

`nvcc` allows the host compiler used for host functions to be specified via the `-ccbin <compiler>` argument. The environment variable `NVCC_CCBIN` can also be defined to specify the host compiler used by `nvcc`. The `-Xcompiler` argument to `nvcc` passes through arguments to the host compiler. For example, in the example below, the `-O3` argument is passed to the host compiler by `nvcc`.

```
nvcc example.cu -ccbin=clang++  
  
export NVCC_CCBIN='gcc'  
nvcc example.cu -Xcompiler=-O3
```

2.5.3.3. Separate Compilation of GPU Code

`nvcc` defaults to *whole-program compilation*, which **expects all GPU code and symbols to be present in the compilation unit that uses them**. CUDA device functions may call device functions or access device variables defined in other compilation units, but either the `-rdc=true` or its alias the `-dc` **flag must be specified** on the `nvcc` command line **to enable linking of device code from different compilation units**. The ability to link device code and symbols from different compilation units is called *separate compilation*.

Separate compilation allows more flexible code organization, can improve compile time, and can lead to smaller binaries. Separate compilation may involve some build-time complexity compared to whole-program compilation. Performance can be affected by the use of device code linking, which is why it is not used by default. [Link-Time Optimization \(LTO\)](#) can help reduce the performance overhead of separate compilation.

Separate compilation requires the following conditions:

- Non-`const` device variables defined in one compilation unit must be referred to with the `extern` keyword in other compilation units.
- All `const` device variables must be defined and referred to with the `extern` keyword.
- All CUDA source files `.cu` must be compiled with the `-dc` or `-rdc=true` flags.

Host and device functions have external linkage by default and do not require the `extern` keyword. Note that starting from CUDA 13, `__global__` functions and `__managed__` / `__device__` / `__constant__` variables have internal linkage by default.

In the following example, `definition.cu` defines a variable and a function, while `example.cu` refers to them. Both files are compiled separately and linked into the final binary.

```
// ----- definition.cu -----  
extern __device__ int device_variable = 5;  
__device__ int device_function() { return 10; }
```

```
// ----- example.cu -----
extern __device__ int device_variable;
__device__ int device_function();

__global__ void kernel(int* ptr) {
    device_variable = 0;
    *ptr            = device_function();
}
```

```
nvcc -dc definition.cu -o definition.o
nvcc -dc example.cu    -o example.o
nvcc definition.o example.o -o program
```

2.5.4. Common Compiler Options

This section presents the **most relevant compiler options** that can be used with `nvcc`, covering language features, optimization, debugging, profiling, and build aspects. The full description of all options can be found in the [nvcc documentation](#).

2.5.4.1. Language Features

`nvcc` supports the C++ core language features, from C++03 to [C++20](#). The `-std` flag can be used to **specify the language standard to use**:

- `--std={c++03|c++11|c++14|c++17|c++20}`

In addition, `nvcc` supports the following **language extensions**:

- `-restrict`: Assert that all kernel pointer parameters are [restrict](#) pointers.
- `-extended-lambda`: Allow `__host__`, `__device__` annotations in lambda declarations.
- `-expt-relaxed-constexpr`: (Experimental flag) Allow host code to invoke `__device__ constexpr` functions, and device code to invoke `__host__ constexpr` functions.

More detail on these features can be found in the [extended lambda](#) and [constexpr](#) sections.

2.5.4.2. Debugging Options

`nvcc` supports the following options to **generate debug information**:

- `-g`: Generate debug information for host code. `gdb/lldb` and similar tools rely on such information for host code debugging.
- `-G`: Generate debug information for device code. [cuda-gdb](#) relies on such information for device-code debugging. The flag also defines the `__CUDACC_DEBUG__` macro.
- `-lineinfo`: Generate line-number information for device code. This option does not affect execution performance and is useful in conjunction with the [compute-sanitizer](#) tool to trace the kernel execution.

`nvcc` **uses the highest optimization level** `-O3` for GPU code by default. The **debug flag** `-G` **prevents some compiler optimizations**, and so **debug code is expected to have lower performance than non-debug code**. The `-DNDEBUG` flag can be defined to disable runtime assertions, as these can also slow down execution.

2.5.4.3. Optimization Options

`nvcc` provides many [options for optimizing performance](#). This section aims to provide a brief survey of some of the options available that developers may find useful, as well as links to further information. Complete coverage can be found in the [nvcc documentation](#).

- `-Xptxas` passes arguments to the PTX assembler tool `ptxas`. The `nvcc` documentation provides a [list of useful arguments](#) for `ptxas`. For example, `-Xptxas=-maxrregcount=N` specifies the maximum number of registers to use, per thread.
- `-extra-device-vectorization`: Enables more aggressive device code vectorization.
- Additional flags which provide fine-grained control over floating point behavior are covered in the [Floating-Point Computation](#) section and in the [nvcc documentation](#).

The [following flags get output](#) from the compiler which can be [useful in more advanced code optimization](#):

- `-res-usage`: Print resource usage report after compilation. It includes the number of registers, shared memory, constant memory, and local memory allocated for each kernel function.
- `-opt-info=inline`: Print information about inlined functions.
- `-Xptxas=-warn-lmem-usage`: Warn if local memory is used.
- `-Xptxas=-warn-spills`: Warn if registers are spilled to local memory.

2.5.4.4. Link-Time Optimization (LTO)

[Separate compilation can result in lower performance](#) than whole-program compilation [due to limited cross-file optimization opportunities](#). Link-Time Optimization (LTO) addresses this by performing optimizations across separately compiled files at link time, at the cost of increased compilation time. LTO [can recover much of the performance of whole-program compilation while maintaining the flexibility of separate compilation](#).

`nvcc` requires the `-dlto` [flag](#) or `lto_<SM version>` link-time optimization targets to enable LTO:

```
nvcc -dc -dlto -arch=sm_100 definition.cu -o definition.o
nvcc -dc -dlto -arch=sm_100 example.cu -o example.o
nvcc -dlto definition.o example.o -o program
```

```
nvcc -dc -arch=lto_100 definition.cu -o definition.o
nvcc -dc -arch=lto_100 example.cu -o example.o
nvcc -dlto definition.o example.o -o program
```

2.5.4.5. Profiling Options

It is [possible to directly profile a CUDA application using the Nsight Compute and Nsight Systems tools](#) without the need for additional flags during the compilation process. However, additional information which can be generated by `nvcc` can assist profiling by correlating source files with the generated code:

- `-lineinfo`: Generate line-number information for device code; this allows viewing the source code in the profiling tools. Profiling tools require the original source code to be available in the same location where the code was compiled.
- `-src-in-ptx`: Keep the original source code in the PTX, avoiding the limitations of `-lineinfo` mentioned above. Requires `-lineinfo`.

2.5.4.6. Fatbin Compression

`nvcc` compresses the [fatbins](#) stored in application or library binaries by default. Fatbin compression can be controlled using the following options:

- `-no-compress`: Disable the compression of the fatbin.
- `--compress-mode={default|size|speed|balance|none}`: Set the compression mode. `speed` focuses on fast decompression time, while `size` aims at reducing the fatbin size. `balance` provides a trade-off between speed and size. The default mode is `speed`. `none` disables compression.

2.5.4.7. Compiler Performance Controls

`nvcc` provides options to analyze and accelerate the compilation process itself:

- `-t <N>`: The number of CPU threads used to parallelize the compilation of a single compilation unit for multiple GPU architectures.
- `-split-compile <N>`: The number of CPU threads used to parallelize the optimization phase.
- `-split-compile-extended <N>`: More aggressive form of split compilation. Requires link-time optimization.
- `-Ofc <N>`: Level of device code compilation speed.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2007-2026, NVIDIA Corporation & affiliates. All rights reserved.

Last updated on Mar 04, 2026.